

WeatherWatch

Capstone Architecture Deep-Dive

S06B01 · Deep-tier track: Networking

Serverless weather poller on AWS — public entry point, external API call, secret at runtime, on-demand storage — reviewed against the Well-Architected Framework and extended into a VPC.

Subahan Talararla · July 2026

1

What's built

Terraform-managed, us-east-1 — all three phases apply cleanly

Public front door

API Gateway HTTP API → Lambda (LabRole).

One real API call

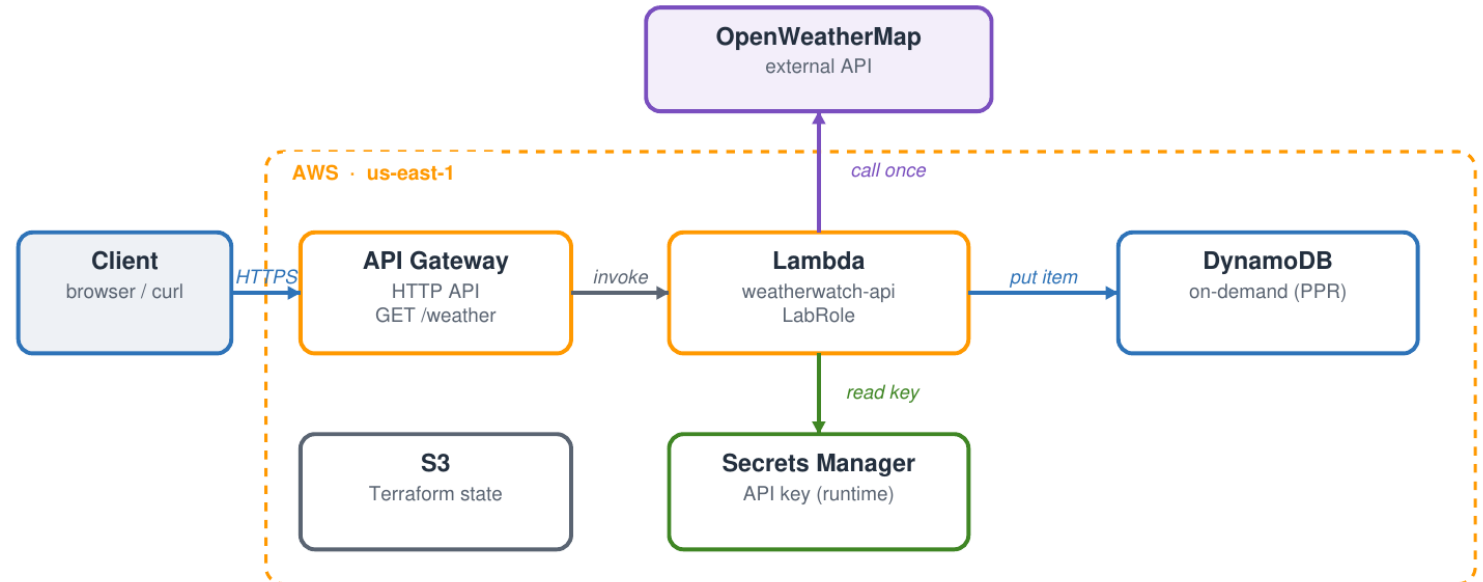
Handler reads the OpenWeatherMap key from Secrets Manager at runtime and calls the API once.

Something real with it

Transforms the response and writes it to an on-demand DynamoDB table.

All as code

Phase 0 table + state, Phase 1 Lambda + secret, Phase 2 front door — nothing hard-coded.



2

What works — shown running

One public request → live weather → persisted in DynamoDB

```
Terminal - live demo

(base) subahantalamarla@Subahans-MacBook-Air phase2-apigateway % curl "$(terraform output -raw weather_endpoint)"
{"message": "Fetched from OpenWeatherMap and stored in DynamoDB", "stored_in":
"weatherwatch-datastore", "item_id": "london#1783608876", "result": {"id":
"london#1783608876", "city": "London", "temp_c": 34.46, "feels_like_c": 34.1,
"humidity": 31, "conditions": "overcast clouds", "fetched_at": 1783608876}}

(base) subahantalamarla@Subahans-MacBook-Air phase2-apigateway % aws dynamodb get-item \
--table-name weatherwatch-datastore \
--key '{"id":{"S":"london#1783608876"}}' --region us-east-1
{ "Item": {
  "city":      {"S": "London"},
  "conditions": {"S": "overcast clouds"},
  "temp_c":    {"N": "34.46"},
  "fetched_at": {"N": "1783608876"},
  "feels_like_c": {"N": "34.1"},
  "humidity":  {"N": "31"},
  "id":        {"S": "london#1783608876"} } }
```

Live demo — run it

terraform output -raw weather_endpoint

curl "\$(...weather_endpoint)"

→ JSON weather, HTTP 200

aws dynamodb get-item ...

→ item persisted

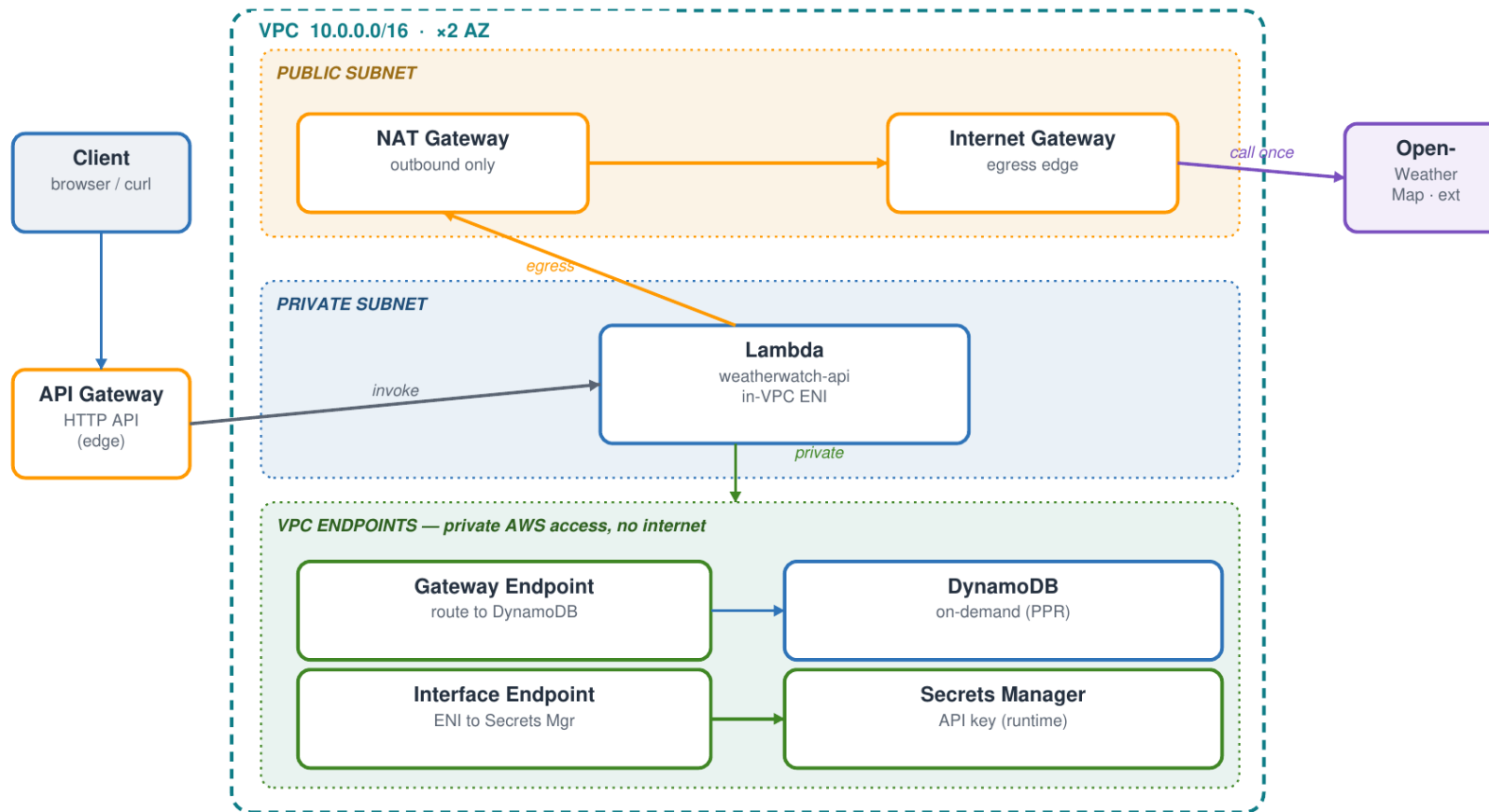
Proven end-to-end

front door • Secrets Manager read • external call •
DynamoDB write — all in one request.

3

Deep-tier: Networking

Target design — Lambda in private subnets, data plane off the public internet



Private compute

Lambda runs in private subnets across 2 AZs — no inbound from the internet.

Private AWS access

VPC endpoints keep DynamoDB (gateway) and Secrets Manager (interface) traffic on the AWS network.

Controlled egress

Only outbound to OpenWeatherMap, via NAT + Internet Gateway.

Pillar: Security

Least exposure — the sensitive paths never touch the public internet.

Decisions × Well-Architected

Every choice named a rejected option and a pillar (6-pillar framework)

COST

On-demand DynamoDB

over RDS

Zero idle cost for a bursty, mostly-idle poller;
RDS wins on ad-hoc SQL but bills 24×7.

OPERATIONAL EXCELLENCE

API Gateway HTTP API

over Function URL (IAM)

Clean public, browser-testable door; a signed
Function URL can't open in a browser.

SECURITY

Secrets Manager

over env var / hard-code

Key stays out of code and state, read at
runtime, rotatable.

Reviewed against all six pillars — Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, Sustainability.

What's left • the one risk

Standup close — the honest gaps and what I'm watching

What's left

- Apply the throttling fix live (it's committed in main.tf).
- Stand up the VPC: private subnets + endpoints + NAT.
- Observability: CloudWatch log retention + X-Ray tracing.
- Least-privilege role in place of shared LabRole (real account).
- Document manual key rotation.

The one risk I'm watching

The public GET /weather route is unauthenticated — every hit spends compute and a paid OpenWeatherMap call, so a burst could run up cost.

Mitigation in flight:

- Stage throttling (10 req/s, burst 5) — committed, returns HTTP 429 past the limit.
- Networking target moves the data plane to private VPC endpoints — off the internet entirely.

Sources

Course references used for this deep-dive — review-round format



AWS Well-Architected Framework (whitepaper, Nov 6 2024)

docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html



Terraform AWS provider — `aws_lambda_function`, `aws_dynamodb_table`, `aws_vpc` / `subnet`, `aws_vpc_endpoint`

registry.terraform.io/providers/hashicorp/aws/latest/docs



AWS Lambda Developer Guide — configuring a function to access a VPC

docs.aws.amazon.com/lambda/latest/dg/configuration-vpc.html



Amazon DynamoDB Developer Guide — on-demand capacity mode

docs.aws.amazon.com/amazondynamodb/latest/developerguide/